

Chip in, big win

UVM Driver 常考题 / 真题汇总

Part 1 / 3 · 题目 1 ~ 12

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 1-12：握手与 Driver 基础形态（题1-6）、配置注入与接口建模（题7-10）、时序与 Reset 处理（题11-12）。

题目 1 Pull Driver 最小可用骨架

难度：初级
标签：get_next_item、item_done、ready/valid、vif、run_phase

【题目描述】

实现一个基于 ready/valid 握手协议的标准 pull driver。

事务类型 req_t
字段：addr（地址）、data（数据）、is_write（读写标志）、byte_en（字节使能）。

接口 vif 信号分为请求侧（req_valid/req_ready/req_addr/req_wdata/req_be）和响应侧（rsp_valid/rsp_rdata/rsp_err）。

完整要求：

1. 使用 get_next_item(req) 与 item_done([rsp]) 正确配对完成握手；

2. 等待 req_ready==1 时才完成握手，期间信号保持稳定（协议要求）；

3. 读事务需采样 rsp_rdata 并构造 rsp 回传给 sequence；

4. 空闲时驱动 idle（valid=0）；reset 到来时拉回默认值，不驱动非法值；

5. 不得出现 0-time busy loop，每次等待必须消耗至少一个时钟边沿。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    // 套用模版：run_phase fork process_trans + monitor_reset
    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    // 套用模版：reset 监测，扩展为拉低所有接口输出
    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
```

```
            v_if.drive_cb.req_valid <= 1'b0;
            v_if.drive_cb.req_addr <= '0;
            v_if.drive_cb.req_wdata <= '0;
            v_if.drive_cb.req_be <= '0;
        end
    endtask

    // 套用模版：process_trans 核心握手循环
    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            if (!req.is_write) begin
                void'($cast(rsp, req.clone()));
                rsp.set_sequence_id(req.get_sequence_id());
                rsp.data = req.data; // read_item 已填入
                seq_item_port.item_done(rsp);
            end else begin
                seq_item_port.item_done();
            end
        end
    endtask

    task drive_item(my_transaction trans);
        if (trans.is_write) write_item(trans);
        else read_item(trans);
    endtask

    // 写：拉高 valid，等 ready 握手完成
    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.req_valid <= 1'b1;
        v_if.drive_cb.req_addr <= trans.addr;
        v_if.drive_cb.req_wdata <= trans.data;
        v_if.drive_cb.req_be <= trans.byte_en;
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.req_ready);
        v_if.drive_cb.req_valid <= 1'b0;
    endtask

    // 读：发地址请求，等 rsp_valid，采样 rsp_rdata
    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.req_valid <= 1'b1;
        v_if.drive_cb.req_addr <= trans.addr;
        v_if.drive_cb.req_be <= trans.byte_en;
```

```
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.req_ready);
        v_if.drive_cb.req_valid <= 1'b0;
        // 等待读响应
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.rsp_valid);
        trans.data = v_if.drive_cb.rsp_rdata;
    endtask
endclass
```

【模版套用说明】

run_phase fork + monitor_reset 结构完整保留；monitor_reset
扩展为同时拉低所有接口输出；process_trans 按读写分支分别回传 rsp；write_item/read_item 与模版对应，read_item 增加响应等待逻辑。

题目 2 get/put_response 风格 Driver

难度：中级
标签：get、put_response、clone/copy、pipeline、安全快照

【题目描述】

实现 get/put_response 握手风格的 driver。

与 get_next_item/item_done 的关键区别：get() 取到 req 后立即解除 sequence 侧 finish_item() 的阻塞，

使 sequence 可以继续产生下一笔事务（实现 pipeline 效果），与 put_response 配对而非 item_done。

完整要求：

1. 使用 seq_item_port.get(req) 代替 get_next_item，不需要配对 item_done；

2. get() 返回后立即对 req 做 clone 快照，防止 sequence 重 randomize
同一对象时污染正在执行的数据；

3. 通过 seq_item_port.put_response(rsp) 回传响应，响应的 sequence_id 必须与原 req 对应；

4. get() 与 get_next_item() 的核心语义差异：get_next_item 会持有 sequencer 直到 item_done，

而 get() 立即释放 sequencer，允许 sequence 继续执行，实现真正的 pipeline 流水。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    // 套用模版：run_phase fork 结构
    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
```

```
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, req_snap, rsp;
        forever begin
            // get() 取到后立即解除 sequence finish_item 阻塞
            seq_item_port.get(req);

            // 关键：立即做快照，防止 sequence 重随机化污染 drive 数据
            $cast(req_snap, req.clone());

            drive_item(req_snap);

            // 构造响应并回传（sequence_id 从原始 req 获取）
            $cast(rsp, req_snap.clone());
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.put_response(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

get() 替代 get_next_item，不调用 item_done，改为 put_response。模版的 process_trans 结构仅改动握手 API，drive_item/read_item/write_item 完全不变。clone() 快照是本题核心，防止对象复用 bug。

题目 3 try_next_item 轮询式 Driver

难度：中级

标签：try_next_item、轮询、item_done 配对、性能、idle

【题目描述】

实现一个基于周期轮询的 driver，每个时钟周期先处理侧带任务，再尝试获取事务。

核心要点（也是常见 bug 点）：

1. 只有 try_next_item(req) 返回非 null 时，才能调用 item_done()；req==null 时调用 item_done 会破坏 sequencer 状态机；

2. 不得用 while(1) 轮询——每次调用 try_next_item 前必须等待时钟边沿，消耗仿真时间，防止 0-time 空转；

3. req==null 时总线保持 idle 状态；

4. 侧带任务（如 credit 更新、心跳）在每拍开始执行，与主事务驱动逻辑正交，封装为独立 task。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
```

```
        my_transaction req;
        forever begin
            // 每拍先等时钟，避免 0-time 空转
            @(v_if.drive_mp.drive_cb);
            // 处理侧带任务（credit 更新、心跳等）
            do_sideband();
            // 尝试非阻塞获取事务
            seq_item_port.try_next_item(req);
            if (req != null) begin
                // 仅在成功获取时才驱动并 item_done
                drive_item(req);
                seq_item_port.item_done();
            end else begin
                // null：保持 idle，绝对不能调用 item_done
                v_if.drive_cb.data <= 8'd0;
            end
        end
    endtask

    task do_sideband();
        // 侧带任务占位：可在此更新 credit 计数或发心跳
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

process_trans 内先 @clk 再 try_next_item，严格区分 null/非null 两个分支，item_done 只在有效 req 分支调用。模版 drive_item/read_item/write_item 不动；monitor_reset 保持原样。侧带任务封装在 do_sideband() 便于扩展。

题目 4 peek+grant：两段式执行 Driver

难度：中级

标签：get_next_item、grant、两阶段时序、timeout、reset

【题目描述】

实现"先观察再执行"的两段式 driver，适用于需要等待总线授权（bus_grant）才能执行的协议。

四个明确阶段：

1. Accept：get_next_item(req) 取到候选事务；

2. 等待 grant：循环等待 bus_grant 信号，支持超时（timeout_cycles 可配置，0=禁用）；

3. 执行：grant 到来后才驱动总线（地址相位、数据相位）；

4. 完成：无论 grant 成功还是超时，都必须调用 item_done 防止 finish_item hang。

Reset 处理：reset 发生时中止等待 grant，立即 item_done 并拉回接口默认值。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    int unsigned timeout_cycles = 1000;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
        void'(uvm_config_db#(int unsigned)::get(
            this, "", "timeout_cycles", timeout_cycles));
    endfunction

    // 套用模版 run_phase fork 结构
    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

```

```
    task process_trans();
        my_transaction req, rsp;
        forever begin
            // 阶段1：Accept
            seq_item_port.get_next_item(req);

            // 阶段2：等待 grant 或超时
            begin
                int cnt = 0;
                bit got_grant = 0;
                while (!v_if.drive_cb.grant) begin
                    @(v_if.drive_mp.drive_cb);
                    if (timeout_cycles > 0 && ++cnt >= timeout_cycles) begin
                        `uvm_error("DRV_TO",
                            $sformatf("Grant timeout after %0d cycles",
                                      timeout_cycles))
                        break;
                    end
                end
                got_grant = v_if.drive_cb.grant;

                // 阶段3：执行（仅 grant 成功时驱动）
                if (got_grant)
                    drive_item(req);
            end

            // 阶段4：完成（无论正常/超时都必须 item_done）
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
```

```
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

两段式结构：get_next_item 后进入 grant 等待循环，超时用计数器控制。无论 grant 是否成功，最后都调用 item_done 防止 hang。模版的 drive_item/read_item/write_item 核心不变。

题目 5 uvm_push_driver：push 模式实现

难度：高级

标签：uvm_push_driver、put、push_sequencer、端口连接、响应

【题目描述】

实现 push 模式 driver，与标准 pull driver 的根本区别在于数据流方向：

push 模式下 sequencer 主动调用 driver.put(req) 推送事务，driver 被动接收并执行。

要求：

1. driver 继承自 uvm_push_driver#(my_transaction)，override put() 虚任务；

2. put() 内完成完整的 pin-level 驱动（阻塞直到本次请求执行完毕）；

3. 可通过 rsp_port.write(rsp) 回传响应（rsp_port 是 push_driver 内建端口）；

4. agent connect_phase 中正确连接：sequencer.req_port -> driver.req_export；

5. 不得调用 seq_item_port 相关 API（push driver 无此端口）。

注意：push driver 无 run_phase 主循环，核心入口是 put() 任务。

【完整答案代码】

```
// push driver：继承 uvm_push_driver，override put()
class my_push_driver extends uvm_push_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_push_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    // push driver 核心入口：sequencer 主动调用 put()
    virtual task put(my_transaction req);
        my_transaction rsp;
        // 驱动接口（与模版 drive_item 风格一致）
        drive_item(req);
        // 通过 rsp_port 回传响应
        $cast(rsp, req.clone());
        rsp.set_sequence_id(req.get_sequence_id());
        rsp_port.write(rsp);
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
```

```
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass

// agent connect_phase 连接示例
class my_agent extends uvm_agent;
    my_push_driver drv;
    uvm_push_sequencer#(my_transaction) sqr;
    `uvm_component_utils(my_agent)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        drv = my_push_driver::type_id::create("drv", this);
        sqr = uvm_push_sequencer#(my_transaction)::type_id::create("sqr", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        // push 模式连接方向：sequencer.req_port -> driver.req_export
        sqr.req_port.connect(drv.req_export);
    endfunction
endclass
```

【模版套用说明】

push driver 继承 uvm_push_driver，核心入口是 put() 而非 run_phase 循环。drive_item/read_item/write_item 与模版完全一致可直接复用。rsp_port 是 push_driver 内建端口，无需额外声明。

题目 6 分层 Driver：上层事务到下层总线

难度：高级

标签：layered、拆分、burst/stream、状态机、响应聚合

【题目描述】

实现分层 driver：接收上层网络包 net_pkt_t，将 payload 按总线位宽（如 32-bit/4 字节）拆分为多个 beat 驱动到 streaming 接口。

拆分要点：

1. beat 数量 = ceil(payload_bytes / BUS_BYTES)；

2. 每个 beat 计算 byte_en（strobe）：非最后一拍全为 1，最后一拍按实际有效字节设位；

3. 最后一个 beat 拉高 last 信号；

4. 每个 beat 等待 ready 握手完成后才推进（支持 backpressure）；

5. 所有 beat 完成后构造上层 rsp_t 并回传；

6. reset 时立即停止拆分，拉低 valid，调用 item_done 防止 hang。

【完整答案代码】

```
class my_driver extends uvm_driver#(net_pkt_t);
    virtual my_interface v_if;
    localparam BUS_BYTES = 4; // 32-bit 总线
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    // 保留模版 fork 结构
    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.valid <= 1'b0;
            v_if.drive_cb.data <= '0;
            v_if.drive_cb.last <= 1'b0;
```

```
        end
    endtask

    task process_trans();
        net_pkt_t req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    // drive_item：把上层 pkt 拆成多个 beat 驱动
    task drive_item(net_pkt_t trans);
        int total_bytes = trans.payload.size();
        int beats = (total_bytes + BUS_BYTES - 1) / BUS_BYTES;
        for (int i = 0; i < beats; i++) begin
            int offset = i * BUS_BYTES;
            int valid_bytes = (total_bytes - offset < BUS_BYTES) ?
                              (total_bytes - offset) : BUS_BYTES;
            bit [31:0] beat_data = '0;
            bit [3:0] beat_be = '0;
            for (int b = 0; b < valid_bytes; b++) begin
                beat_data[b*8 +: 8] = trans.payload[offset + b];
                beat_be[b] = 1'b1;
            end
            write_item_beat(beat_data, beat_be, (i == beats - 1));
        end
        v_if.drive_cb.valid <= 1'b0;
    endtask

    // 单个 beat 驱动，等待 ready 握手
    task write_item_beat(
        input bit [31:0] data,
        input bit [3:0] be,
        input bit last
    );
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.valid <= 1'b1;
        v_if.drive_cb.data <= data;
        v_if.drive_cb.byte_en <= be;
        v_if.drive_cb.last <= last;
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.ready);
    endtask
```

```
endclass
```

【模版套用说明】

模版 process_trans 骨架完全保留；drive_item 替换为 pkt 拆分循环，write_item 扩展为带 be/last 参数的 write_item_beat。strobe 按实际有效字节位计算，最后一拍才置 last。monitor_reset 额外清零 last 信号。

题目 7 config_db 注入 virtual interface

难度：初级

标签：uvm_config_db、virtual interface、build_phase、错误处理、复用

【题目描述】

实现 virtual interface 通过 config_db 精确注入到多个 driver 实例的标准方案。

关键原则——"范围最小化"：

1. top 中有两个 interface 实例 if0/if1，分别对应 agent0 和 agent1；

2. top 中的 set() 路径精确到 agent0.* / agent1.*，不使用全局 * 通配符；

3. driver 的 build_phase 中用 get(this, "", "vif", v_if) 获取，路径 "" 表示从自身开始向上查找；

4. get() 失败时立即 uvm_fatal，并打印 get_full_name() 提供精确的层次路径，便于快速定位；

5. driver 代码本身不含路径硬编码，可扩展到 N 个 agent 而无需修改；

6. set/get 的类型必须完全一致（virtual my_interface）。

【完整答案代码】

```
// === top module：精确路径注入两套 vif ===
module top;
    my_interface if0(clk, reset_n);
    my_interface if1(clk, reset_n);

    initial begin
        // 精确注入：不使用全局 * 通配，范围最小化
        uvm_config_db#(virtual my_interface)::set(
            null, "uvm_test_top.env.agent0.*", "vif", if0);
        uvm_config_db#(virtual my_interface)::set(
            null, "uvm_test_top.env.agent1.*", "vif", if1);
        run_test("my_test");
    end
endmodule

// === driver：build_phase 正确获取 vif ===
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        // get 失败时 uvm_fatal，打印完整层次路径
        if (!uvm_config_db#(virtual my_interface)::get(
                this, "", "vif", v_if))
            `uvm_fatal("NO_VIF",
```

```
                $sformatf("[%s] Failed to get 'vif' from config_db",
                          get_full_name()))
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

set() 路径精确到 agent0.* / agent1.*，避免 * 全局污染导致串线。driver 侧 get() 用 (this, "") 即可匹配父级 set 路径。uvm_fatal 打印 get_full_name() 便于快速定位。模版其余部分不变。

题目 8 多实例 Agent 的差异化配置

难度：中级
标签：config_db、wildcard、precedence、cfg object、可扩展

【题目描述】

实现多实例 agent 的差异化配置方案，环境中有 agent[0:7]，每个 driver 需要不同的 base_addr 与 timeout_cycles。

设计要点：

1. 定义配置类 my_driver_cfg（含 base_addr、timeout_cycles 字段，使用 uvm_object_utils 注册）；

2. env 的 build_phase 中用 for 循环为每个 agent 创建独立 cfg 对象（不能共享同一个句柄，否则所有 agent 会看到最后一次赋值）；

3. set() 路径精确到每个 driver 实例（$sformatf("agent[%0d].driver", i)）；

4. config_db precedence 规则：路径更精确的 set 优先；同等精度时后 set 的覆盖先 set 的；

5. driver build_phase 中 get() 配置，失败时使用默认值并打印 warning（不 fatal，保持健壮性）；

6. 方案线性扩展——N 个 agent 只需修改循环上限。

【完整答案代码】

```
// === 配置类定义 ===
class my_driver_cfg extends uvm_object;
    `uvm_object_utils_begin(my_driver_cfg)
        `uvm_field_int(base_addr, UVM_ALL_ON)
        `uvm_field_int(timeout_cycles, UVM_ALL_ON)
    `uvm_object_utils_end
    rand bit [31:0] base_addr;
    int unsigned timeout_cycles = 200;
endclass

// === env build_phase：批量注入（每实例独立 cfg 对象）===
function void build_phase(uvm_phase phase);
    for (int i = 0; i < 8; i++) begin
        my_driver_cfg cfg;
        cfg = my_driver_cfg::type_id::create($sformatf("cfg%0d", i));
        cfg.base_addr = 32'h1000_0000 + i * 32'h1000;
        cfg.timeout_cycles = 100 + i * 10;
        uvm_config_db#(my_driver_cfg)::set(
            this,
            $sformatf("agent[%0d].driver", i),
            "cfg",
```

```
            cfg);
    end
endfunction

// === driver：套用模版 + 获取 cfg ===
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    my_driver_cfg cfg;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
        if (!uvm_config_db#(my_driver_cfg)::get(this, "", "cfg", cfg)) begin
            `uvm_warning("NO_CFG", "No cfg found, using defaults")
            cfg = my_driver_cfg::type_id::create("cfg");
        end
        `uvm_info("DRV_CFG",
            $sformatf("[%s] base_addr=0x%0h timeout=%0d",
                get_full_name(), cfg.base_addr, cfg.timeout_cycles),
            UVM_MEDIUM)
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
```

```
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

for 循环批量 set，每个 agent 独立 cfg 对象（不共享句柄）。driver 侧 build_phase 多一次 cfg get，get 失败用 warning 而非 fatal 保证健壮性。模版其余部分不变。

题目 9 Driver 使用 modport + Clocking Block

难度：中级

标签：modport、clocking block、竞态消除、vif 类型、信号方向

【题目描述】

设计 modport + clocking block 组合消除驱动竞态，实现接口权限隔离。

核心原理：

1. clocking block output #1：驱动信号在 posedge clk 后 1 个时间单位发生，避免 DUT 在同一边沿采到不确定值；

2. clocking block input #1step：在 posedge clk 前 1step 采样，避免竞争；

3. drv_mp 绑定 cb_drv，赋予 driver 对输出信号的写权限和输入信号的读权限；

4. mon_mp 只读，防止 monitor 误写信号；

5. driver 声明 virtual my_interface.drv_mp vif，通过 @(v_if.cb_drv) 等待时钟并用 <= 非阻塞驱动输出；

6. 禁止在 @posedge clk 内用 blocking 赋值（=）驱动 DUT 输入，会与 DUT 采样产生竞争。

【完整答案代码】

```
// === Interface 定义：modport + clocking block ===
interface my_interface(input logic clk, reset_n);
    logic req_valid, req_addr, req_data;
    logic rsp_valid, rsp_data;

    // driver clocking block：output #1 消除竞态，input #1step 采样
    clocking cb_drv @(posedge clk);
        default input #1step output #1;
        output req_valid, req_addr, req_data;
        input rsp_valid, rsp_data;
    endclocking

    // monitor clocking block：只读
    clocking cb_mon @(posedge clk);
        default input #1step;
        input req_valid, req_addr, req_data, rsp_valid, rsp_data;
    endclocking

    // modport 权限控制
    modport drv_mp(clocking cb_drv, input clk, reset_n);
    modport mon_mp(clocking cb_mon, input clk, reset_n);
endinterface

// === Driver：使用 modport 类型，通过 cb_drv 无竞态驱动 ===
```

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface.drv_mp v_if; // modport 绑定类型
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface.drv_mp)::get(
                this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "drv_mp vif not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.reset_n);
            v_if.cb_drv.req_valid <= 1'b0;
            v_if.cb_drv.req_data <= '0;
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.cb_drv);
        trans.data = v_if.cb_drv.rsp_data;
    endtask

    task write_item(my_transaction trans);
```

```
        @(v_if.cb_drv);
        v_if.cb_drv.req_valid <= 1'b1;
        v_if.cb_drv.req_data <= trans.data;
    endtask
endclass
```

【模版套用说明】

clocking block output #1 确保驱动发生在 clk 边沿后 1 个时间单位，避免 DUT 采样不确定值。模版的 drive_mp.drive_cb 替换为 cb_drv，其余结构不变。config_db 的类型也需要改为 virtual my_interface.drv_mp。

题目 10 Active/Passive 可切换 Driver 门控

难度：中级

标签：active/passive、agent、门控、避免 hang、配置

【题目描述】

实现 active/passive 可切换的 agent，推荐方案是在 agent 层门控 driver/sequencer 的创建而非在 driver 内部加 if 判断。

推荐方案（最干净）：

1. agent.build_phase 中调用 get_is_active()，仅 UVM_ACTIVE 时才创建 driver 和 sequencer；

2. passive 时只创建 monitor（monitor 本身不消耗 seq item，不会造成 hang）；

3. connect_phase 同样门控：仅 active 时连接 seq_item_port；

4. is_active 通过 uvm_config_db 从 top/test 注入；

5. driver 代码本身不需要任何 is_active 相关判断，逻辑完全在 agent 层处理，driver 代码直接套用模版。

【完整答案代码】

```
// === agent：在 build_phase 层门控 driver/sequencer 创建 ===
class my_agent extends uvm_agent;
    my_driver drv;
    my_sequencer sqr;
    my_monitor mon;
    `uvm_component_utils(my_agent)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        // monitor 总是创建（passive 模式仍需观测）
        mon = my_monitor::type_id::create("mon", this);
        // 仅 active 时创建 driver 和 sequencer
        if (get_is_active() == UVM_ACTIVE) begin
            drv = my_driver::type_id::create("drv", this);
            sqr = my_sequencer::type_id::create("sqr", this);
        end
    endfunction

    function void connect_phase(uvm_phase phase);
        if (get_is_active() == UVM_ACTIVE)
            drv.seq_item_port.connect(sqr.seq_item_export);
    endfunction
endclass

// === driver：完全套用模版，无需感知 is_active ===
```

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
```

```
    endtask
endclass

// === top：注入 passive 配置示例 ===
// uvm_config_db#(uvm_active_passive_enum)::set(
// null, "uvm_test_top.env.agent1", "is_active", UVM_PASSIVE);
```

【模版套用说明】

最佳实践是在 agent.build_phase 用 get_is_active() 门控 driver/sequencer 的创建。driver 代码完全套用模版不需改动——passive 时根本不会被实例化，也就不存在 hang 问题。

题目 11 初始 Reset 等待 + 运行中 Reset 保护

难度：高级

标签：reset、事务中止、恢复策略、默认值、健壮性

【题目描述】

实现同时支持初始 reset 等待和运行中 reset 保护的完整 driver。

两种 reset 场景的处理：

1. 初始 reset：run_phase 开头用 wait(reset === 1) 等待释放，期间不 pull item；

2. 运行中 reset：在外层 forever 循环中，每次 fork process_trans + monitor_reset 用 join_any 运行，

monitor_reset 检测到 reset 下沿后立即返回，join_any 触发后 disable fork 终止 process_trans；

3. 中止时：调用 drive_idle() 拉回所有接口信号到默认值；

4. reset 结束后（reset_n 重新拉高）：自动重新进入 fork 循环继续工作；

5. 被 disable fork 杀掉的事务，通过 monitor_reset 中的 uvm_info 记录中止点。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        // 初始等待 reset 释放
        wait(v_if.mon_cb.reset === 1'b1);
        @(v_if.drive_mp.drive_cb); // 对齐时钟边沿

        forever begin
            // fork process_trans + monitor_reset，reset 到来时 join_any 触发
            fork
                process_trans();
                monitor_reset();
            join_any
            disable fork; // 终止所有子线程

            // 拉回接口到安全默认值
            drive_idle();
```

```
            `uvm_info("DRV", "Reset asserted: bus driven to idle", UVM_LOW)

            // 等待 reset 释放，然后重新开始
            wait(v_if.mon_cb.reset === 1'b1);
            @(v_if.drive_mp.drive_cb);
            `uvm_info("DRV", "Reset released: resuming", UVM_LOW)
        end
    endtask

    task monitor_reset();
        // reset 下沿时立即返回，触发上层 join_any
        @(negedge v_if.mon_cb.reset);
        `uvm_info("DRV", "Reset detected in monitor_reset", UVM_MEDIUM)
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_idle();
        v_if.drive_cb.data <= 8'd0;
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

run_phase 外层 forever + fork/join_any + disable fork 实现可重入 reset：monitor_reset 检测到 reset 下沿后返回，join_any 触发，disable fork 杀掉 process_trans，drive_idle 清信号，等 reset 释放后重新 fork。模版的 process_trans 结构不变。

题目 12 On-the-fly Reset：显式状态机

难度：高级

标签：on-the-fly reset、状态机、资源释放、异常路径、可恢复性

【题目描述】

实现显式状态机的 on-the-fly reset driver，使 reset 处理有迹可循、资源释放有条不紊。

状态枚举：IDLE -> ADDR -> DATA -> RSP -> (遇到 reset) -> RESET -> IDLE。

设计要点：

1. 每个协议阶段结束后检查 state == ST_RESET，决定是否早退；

2. monitor_reset 线程负责状态迁移和接口 idle，不直接 disable fork；

3. 已取到的 item（get_next_item 成功）在 RESET 路径上也必须有对应的 item_done，防止 finish_item hang；

4. semaphore 在 monitor_reset 中重置，避免 process_trans 挂在 sem.get() 上；

5. 状态机让调试时（uvm_info 打印状态）一目了然，scoreboard
可以通过状态判断事务是否有效。

【完整答案代码】

```
typedef enum {ST_IDLE, ST_ADDR, ST_DATA, ST_RSP, ST_RESET} drv_state_e;

class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    drv_state_e state = ST_IDLE;
    semaphore bus_sem;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        bus_sem = new(1);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    // 套用模版 fork 结构
    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    task monitor_reset();
```

```
        forever begin
            @(negedge v_if.mon_cb.reset);
            state = ST_RESET;
            // 重置 semaphore，防止 process_trans 挂在 get 上
            while (!bus_sem.try_get()) ;
            bus_sem.put();
            v_if.drive_cb.data <= 8'd0;
            `uvm_info("DRV", "on-the-fly reset: state -> ST_RESET", UVM_LOW)
            @(posedge v_if.mon_cb.reset);
            state = ST_IDLE;
            `uvm_info("DRV", "reset released: state -> ST_IDLE", UVM_LOW)
        end
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            state = ST_IDLE;
            seq_item_port.get_next_item(req);
            if (state == ST_RESET) begin
                seq_item_port.item_done(); continue;
            end

            state = ST_ADDR;
            @(v_if.drive_mp.drive_cb);
            if (state == ST_RESET) begin
                seq_item_port.item_done(); continue;
            end

            state = ST_DATA;
            drive_item(req);
            if (state == ST_RESET) begin
                seq_item_port.item_done(); continue;
            end

            state = ST_RSP;
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
```

```
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

状态机枚举让调试时一目了然。每个阶段后检查 state==ST_RESET
决定是否早退，保证每个已取到的 item 都有 item_done。monitor_reset 只管状态迁移和拉 idle，不直接 disable fork，资源释放更干净。